

R5.09 Virtualisation avancée

Michaël Hauspie¹

Plan du cours

1 Semaine 1

- Présentation du cours
- Rappels
- Cloud computing et son vocabulaire
- Ce cours
- Vagrant
- Vagrant, réseau
- Environnement à plusieurs machines

Équipe

- Bruno Beaufiles
- Michaël Hauspie
- Yvan Peter

Organisation

- 4 semaines
- 1h de cours
- 3h de TP

Évaluation

- Une séance d'évaluation
 - un QCM sans document
 - Un CTP sans votre session, sans internet et sans document sauf la documentation officielle des outils

Clés d'inscription

- Groupe A-M : 5qmtz8
- Groupe A-N : t7fy2u
- Groupe A-O : t2vgqs
- Groupe B-L : pjvvh3

Contenu

- Rappel rapide sur la virtualisation
- Généralité, cloud
- Vagrant
- Terraform/OpenTofu

Plan du cours

1 Semaine 1

- Présentation du cours
- **Rappels**
- Cloud computing et son vocabulaire
- Ce cours
- Vagrant
- Vagrant, réseau
- Environnement à plusieurs machines

Qu'est ce la virtualisation

- Une abstraction d'un matériel
- Un moyen d'isoler plusieurs applications/systèmes
- Un moyen de partager une même ressource physique

La virtualisation, pourquoi ?

- Exécuter un logiciel sur un matériel pour lequel il n'a pas été prévu (émulation)
- Optimiser les ressources
- Isoler les services pour apporter sécurité et stabilité
- S'assurer que l'environnement de développement et de production identique/compatible

Quelques définitions

Hyperviseur

Logiciel qui permet d'exécuter plusieurs machines virtuelles sur une seule machine physique

Machine hôte

La machine qui **exécute** l'hyperviseur. On peut décliner la définition :

- système d'exploitation hôte
- architecture hôte
- ...

Machine invitée

la machine virtuelle qui **est exécutée** par l'hyperviseur. On peut également décliner la définition :

- système d'exploitation invité
- architecture invitée
- ...

Plan du cours

1 Semaine 1

- Présentation du cours
- Rappels
- Cloud computing et son vocabulaire
- Ce cours
- Vagrant
- Vagrant, réseau
- Environnement à plusieurs machines

C'est quoi le « cloud »

- Un terme générique et assez nébuleux
- Tout ce qui permet d'externaliser une infrastructure, une application, un service...
- *L'ordinateur de quelqu'un d'autre*

Quelques principes/caractéristiques

- Les ressources (calcul, stockage, réseau. . .) sont disponibles à la demande, en libre service
- Elles sont mutualisées (partage des ressources matériels à plusieurs utilisateurs)
- Le paiement est généralement fait à l'usage

Plusieurs modèles de service

En fonction des besoins, des compétences de l'utilisateur et des contraintes sur les traitements de données, on peut utiliser plusieurs modèles de services cloud

- Pas de cloud, hébergement local
- **IaaS**, *Infrastructure as a Service*
- **PaaS**, *Platform as a Service*
- **SaaS**, *Software as a Service*

Hébergement local

On utilise pas d'opérateur cloud, on héberge en local et on supporte le coût de la mise en œuvre et du matériel

- Avantages : on conserve le contrôle complet de ses données et de ses traitements ainsi qu'une complète indépendance
- Inconvénients : nécessite des compétences, des locaux, de l'organisation...
 - Ingénieurs/Techniciens qualifiés
 - Salle serveur équipée
 - Redondance électrique, réseau, stockage
 - ...

IaaS : *Infrastructure as a Service*

L'opérateur cloud fournit des ressources d'infrastructure, machines virtuelles, espace de stockage brut, serveurs physiques, équipements réseau

■ Avantages

- on conserve le contrôle des données (si on chiffre et qu'on maintient une politique de sauvegarde correcte)
- on s'affranchit de la gestion du matériel : pas de locaux, pas de logistique, pas de manutention. . .

■ Inconvénients

- nécessite toujours des compétences importantes
- introduit une dépendance vis-à-vis d'un intervenant tiers (vrai pour tous les modèles cloud)

Quelques acteurs IaaS : OVHcloud, Scaleway, AWS, GCP, Cloudflare, Digital Ocean. . .

PaaS : *Platform as a Service*

L'opérateur cloud ne fournit pas un accès à un serveur (pas de compte root donc), mais à un environnement support à l'exécution d'applications développée par l'utilisateur

- Avantages

- Pas de compétence système et réseau nécessaire
- On peut se concentrer uniquement sur le développement de l'application

- Inconvénients

- On s'enferme dans une technologie
- Moins de maîtrise sur l'environnement de déploiement

Quelques acteurs PaaS : Microsoft Azure, Vercel, Heroku, Google App Engine, AWS...

SaaS : *Software as a Service*

L'opérateur cloud fournit une application clé en main

- Avantages

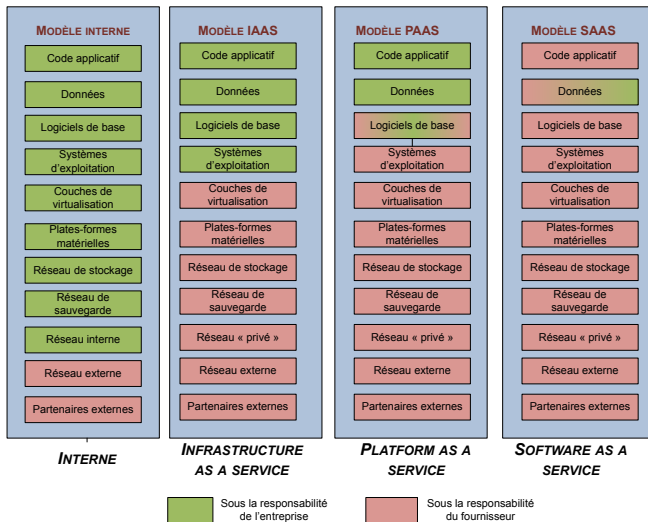
- Pas de compétences en développement nécessaire non plus
- On ne s'occupe plus de l'informatique

- Inconvénients

- On est encore plus enfermé dans une application
- Et encore plus dépendant de l'opérateur
- On a pas de contrôle sur les données et sur ce qu'en fait l'opérateur

Quelques exemples de SaaS : Discord, Office 365, Zoom, Teams, SAP...

Vue d'ensemble



Source Wikimedia, PhFabre, sous license CC-BY-SA 3.0)

Plan du cours

1 Semaine 1

- Présentation du cours
- Rappels
- Cloud computing et son vocabulaire
- **Ce cours**
- Vagrant
- Vagrant, réseau
- Environnement à plusieurs machines

On va y faire quoi, dans ce cours ?

Étudier deux solutions d'automatisation de création d'infrastructure

Vagrant

Pour le développement

- créer facilement plusieurs VM pour développer ses applications
- pas pour la production

Terraform/OpenTofu

Contexte **IaaS** pour le déploiement

- Automatiser la création d'une infrastructure sur un fournisseur **IaaS**

Plan du cours

1 Semaine 1

- Présentation du cours
- Rappels
- Cloud computing et son vocabulaire
- Ce cours
- **Vagrant**
- Vagrant, réseau
- Environnement à plusieurs machines

Présentation de vagrant

- Vagrant est un outil qui permet de facilement mettre en place un environnement de développement à base de machine virtuelle
- Contrairement à docker, le but est de virtualiser des machines
- A utiliser pour le **développement**. Pour la production, on préférera **Terraform/OpenTofu** que nous verrons ensuite

Principe

- Dans un fichier `Vagrantfile`, on décrit l'infrastructure que l'on souhaite mettre en place et comment on va l'initialiser. C'est en réalité un script ruby
- Vagrant va utiliser un fournisseur de virtualisation (*provider*) pour instancier l'infrastructure que l'on souhaite
- En TP, nous utiliseront VirtualBox, mais Vagrant supporte aussi d'autres fournisseurs

- Les VM sont basées sur des modèles : les *boxes*
 - <https://portal.cloud.hashicorp.com/vagrant/discover>
- **Attention** Hashicorp planifie l'arrêt de l'hébergement de *boxes* public. A terme (2027), il faudra créer et héberger ses propres *boxes*

Exemple de Vagrantfile minimal

```
Vagrant.configure("2") do |config|  
  config.vm.box = "cloud-image/debian-13"  
end
```

Quelques commandes

- `vagrant init` : Crée un fichier `Vagrantfile` de base
- `vagrant up` : instancie l'infrastructure décrite dans le fichier `Vagrantfile`
- `vagrant halt` : arrête les machines virtuelles de l'infrastructure
- `vagrant destroy` : détruit l'infrastructure
- `vagrant status` : affiche l'état de l'infrastructure
- `vagrant ssh` : se connecte sur l'une des machines virtuelle de l'infrastructure

Un exemple un peu plus complexe 1/2

```
Vagrant.configure("2") do |config|
  config.vm.box = "cloud-image/debian-13"

  # ajout de fichier dans les VM
  config.vm.provision "file", source: "monfichier_local", destination: "/mon/fichier/dans/la/vm"

  # execution d'une commande dans les VM
  config.vm.provision "shell", inline: "touch /unfichier"

  # Une première VM qui appartient à deux réseau privé et pourra
  # servir de routeur
  config.vm.define "vm1" do |vm1|

    vm1.vm.hostname = "vm1"

    # Les deux réseaux
    vm1.vm.network "private_network", ip: "192.168.33.1"
    vm1.vm.network "private_network", ip: "192.168.32.1"

    # On active le routage
    vm1.vm.provision "shell",
      inline: "echo 1 > /proc/sys/net/ipv4/ip_forward",
      run: "always"
  end

  # ... suite page suivante
```

Un exemple un peu plus complexe 2/2

...

Une deuxième VM qui n'est que dans le réseau 1

```
config.vm.define "vm2" do |vm2|
```

```
  vm2.vm.hostname = "vm2"
```

Sur le réseau 1

```
  vm2.vm.network "private_network", ip: "192.168.32.11"
```

Et on ajoute la route vers le réseau 2

```
  vm2.vm.provision "shell",  
    inline: "ip route add 192.168.33.0/24 via 192.168.32.1",  
    run: "always"
```

end

Une troisième VM qui n'est que dans le réseau 2

```
config.vm.define "vm3" do |vm3|
```

```
  vm3.vm.hostname = "vm3"
```

Sur le réseau 2

```
  vm3.vm.network "private_network", ip: "192.168.33.11"
```

Et on ajoute la route vers le réseau 1

```
  vm3.vm.provision "shell",  
    inline: "ip route add 192.168.32.0/24 via 192.168.33.1",  
    run: "always"
```

end

end

Cycle de vie de l'environnement

- `vagrant up` : démarrage de l'environnement. S'il n'existe pas, l'environnement est créé
- `vagrant halt` : stoppe l'environnement. Arrête uniquement les VM, ne les détruit pas
- `vagrant destroy` : stoppe l'environnement et le supprime
- `vagrant reload` : équivalent de `vagrant halt` suivi de `vagrant up`. Utile pour prendre en compte une modification du `Vagrantfile`

Provisionnement

Le provisionnement est l'ensemble des actions qui permettent la configuration de l'environnement d'une machine (VM ici) au démarrage :

- personnalisation de la VM au delà de la configuration de l'image de base
- processus automatique
 - pas d'interaction
- un simple vagrant up pour avoir un environnement prêt à utiliser

Le provisionnement dans Vagrant

Vagrant gère le provisionnement à l'aide de *provisionners*

- chaque *provisionner* permet une action particulière
 - ajouter un fichier
 - exécuter un script
 - utiliser un outil d'automatisation évolué
 - Ansible
 - Chef
 - Puppet
 - ...

Exécution du provisionnement

Le provisionnement a lieu :

- lors du premier `vagrant up`. Si l'environnement existe déjà, le provisionnement n'est pas exécuté
- lors de l'exécution de `vagrant provision`
- en utilisant l'option `--provision` avec les commandes `vagrant up` et `vagrant reload`

Définition du provisionnement

```
Vagrant.configure("2") do |config|  
  # ...  
  
  config.vm.provision <PROVISIONER>, <PARAMETRES>  
end
```

Provisionnement par copie de fichier

Le provisionneur "file" permet de copier un fichier de la machine physique aux machines virtuelles

```
Vagrant.configure("2") do |config|  
  # ...  
  
  config.vm.provision "file",  
    source: "~/.bashrc",  
    destination: ".bashrc"  
  
end
```

Provisionnement par exécution de script

Le provisionneur `shell` permet d'exécuter un script `shell`

- le script peut provenir d'un fichier local (`path`)
- ou être donné directement dans le fichier `Vagrantfile` (`inline`)
- nombreuses options supplémentaires disponibles

```
Vagrant.configure("2") do |config|  
  # ...  
  
  config.vm.provision  
    "shell",  
    inline: "apt-get update && apt-get -y install vim"  
end
```

Plan du cours

1 Semaine 1

- Présentation du cours
- Rappels
- Cloud computing et son vocabulaire
- Ce cours
- Vagrant
- **Vagrant, réseau**
- Environnement à plusieurs machines

Vagrant permet de définir des environnements réseaux complexes

- redirection de ports
 - rendre disponible un service de la VM sur l'hôte
- réseaux multiples
- configuration automatique ou non

Redirection de ports

```
Vagrant.configure("2") do |config|  
  config.vm.network "forwarded_port", guest: 80, host: 8080  
end
```

Redirige TCP par défaut

- changeable avec le paramètre `protocol` ("tcp" ou "udp")

Réseaux privés

- Réseau de communication entre les VM
 - non disponibles à l'extérieur de l'hôte (au moins en principe, dépend du *provider* de virtualisation)
- On peut définir plusieurs réseaux
- Équivalent à un domaine de collision

Configuration statique

```
Vagrant.configure("2") do |config|  
  config.vm.network "private_network", ip: "192.168.50.4"  
end
```

DHCP

```
Vagrant.configure("2") do |config|  
  config.vm.network "private_network", type: "dhcp"  
end
```

La signification exacte dépend du provider

- généralement la VM est *attachée* au réseau physique de la machine hôte
- La VM est disponible sur le réseau physique de la machine hôte, et donc également aux autres machines du réseau

Attention : ne pas utiliser cette configuration en TP

- Chaque prise des commutateurs des salles de TP n'accepte **que** la machine de TP qui y est branchée
- Si une autre carte est branchée, le commutateur coupe le port

Plan du cours

1 Semaine 1

- Présentation du cours
- Rappels
- Cloud computing et son vocabulaire
- Ce cours
- Vagrant
- Vagrant, réseau
- Environnement à plusieurs machines

Plusieurs machines

config.vm.define permet de créer une nouvelle machine

```
Vagrant.configure("2") do |config|
```

```
  # Un provisionnement exécuté sur les deux machines
```

```
  config.vm.provision "shell", inline: "echo Hello"
```

```
  config.vm.define "web" do |web|
```

```
    web.vm.box = "apache"
```

```
  end
```

```
  config.vm.define "db" do |db|
```

```
    db.vm.box = "mysql"
```

```
  end
```

```
end
```

Plusieurs machines

- `config.vm.define` définit un nouvel objet ruby, de même type que l'objet `config`
- Pour chaque machine, le nouvel objet est fusionné avec l'objet `config`

Provisionnement et multi-machines

```
Vagrant.configure("2") do |config|  
  config.vm.box = "cloud-image/debian-13"  
  config.vm.provision :shell, inline: "echo A"  
  
  config.vm.define :testing do |test|  
    test.vm.provision :shell, inline: "echo B"  
  end  
  
  config.vm.provision :shell, inline: "echo C"  
end
```

- L'ordre d'exécution dépend du niveau d'imbrication
- Le provisionnement global s'exécute avant le provisionnement local
- Ici : A, puis C et enfin B